Methodology

1. Overview

The forecasting task is to predict, at every hour, the all-sky surface shortwave downward irradiance (ALLSKY_SFC_SW_DWN, W/m²) for each of the following 24 hours at five sites in Türkiye. Point forecasts are produced by a Long Short-Term Memory (LSTM) network, and predictive uncertainty is quantified by combining a moving-block bootstrap ensemble with Monte Carlo (MC) Dropout. The combination follows the hybrid ensemble–dropout scheme of Rajaperumal and Chinnappan, adapted in three respects: an LSTM replaces their physics-constrained feed-forward backbone, irradiance replaces photovoltaic power as the target, and the bootstrap resamples contiguous blocks of the training series rather than individual observations.

The procedure consists of the following stages, each described below. The terms *period*, *window*, *block*, *replicate* and *pass* have precise meanings, defined in Section 2.

1. Split the record chronologically into a training, a validation and a test period, and standardise every variable using the training period only.
2. Cut each site's series into overlapping windows of 48 hours: 24 input hours followed by the 24 hours to be forecast.
3. Build B bootstrap replicates of the training windows by drawing blocks of consecutive windows with replacement.
4. Train one LSTM on each replicate, with dropout.
5. Run every trained network M times on each test window with dropout kept active (MC Dropout), giving $B \times M$ forecasts per window.
6. Pool these forecasts into a predictive distribution, from which the point forecast, the variance decomposition and the 95% prediction interval are derived.
7. Set forecasts to zero when the sun is below the horizon.
8. Optionally recalibrate the interval width by split-conformal prediction.
9. Evaluate on the test period.

2. Terminology: hours, periods, windows, blocks, replicates and passes

A single trained network gives a single forecast for each hour, for example 650 W/m² at 12:00 tomorrow, and says nothing about how certain that forecast is. The approach taken here is to produce many slightly different forecasts for the same hour and to measure how much they disagree: if they all lie close to 650 W/m² the model is confident, and if they range from 350 to 1000 W/m² it is not. Two independent mechanisms create the differences:

- **Resampling (the bootstrap)** changes the *data* a network learns from. Several training sets, called *replicates*, are drawn at random from the original one, and a separate network is trained on each.
- **MC Dropout** changes the *network* at prediction time. Each trained network is run repeatedly on the same input with a different random subset of its neurons switched off each time; each run is a *pass*.

The bootstrap and MC Dropout therefore operate on different objects, and the parameters that control them are easy to confuse. In particular, the 24-hour input length of a window and the 168-window length of a bootstrap block are unrelated settings, and MC Dropout involves neither blocks nor resampling.

2.1 Definitions

Table 1. Terms used throughout, from the smallest unit to the largest. Counts are per site unless stated otherwise.

Term	Definition	Size in this study	Used by
Hour	One hourly record of one site, in local solar time	61,368 per site	All stages
Period	One of three consecutive, non-overlapping date ranges: training, validation, test	45,412 / 6,750 / 9,206 hours	All stages
Window	48 consecutive hours of one site: 24 input hours, then 24 target hours	45,365 / 6,703 / 9,159 windows	Training and prediction
Forecast origin	The last input hour of a window, i.e. the hour at which the forecast is issued	One per window	–
Horizon step h	One of the 24 target hours of a window, $h = 1$ being the hour after the origin	24 per window	Evaluation
Element	One window at one horizon step; the unit every metric is averaged over	219,816 test elements	Evaluation
Block	ℓ consecutive training windows of one site	ℓ = 168 windows	Bootstrap only
Replicate	A resampled copy of the training windows, assembled from blocks	$B = 8$	Bootstrap only
Network	One LSTM, trained on one replicate	$B = 8$	–
Pass	One forward run of a network on a window with a fresh random dropout mask	$M = 100$ per network	MC Dropout only

2.2 Periods

Each site's hourly series is cut at two dates that are the same for all sites:

```
 training period (74%)            validation period (11%)      test period (15%)
|------------------------------|----------------------------|-------------------------|
30 Jun 2019 00:00    3 Sep 2024 03:00             11 Jun 2025 09:00        29 Jun 2026 23:00
      45,412 hours                       6,750 hours                   9,206 hours
```

The training period is used to fit the standardisation, to build the bootstrap replicates and to train the networks. The validation period is used for early stopping, learning-rate scheduling and, when used, conformal recalibration. The test period is used only for evaluation.

2.3 Windows

A window is the unit the network sees: the 24 most recent hours as input, and the following 24 hours as the target.

```
hour:     t−23  t−22  ...  t−1   t   | t+1   t+2   ...  t+24
          <------ 24 input hours ----> <---- 24 target hours ---->
                                    ^
                              forecast origin t
```

Windows are taken at every hour (stride 1), so consecutive windows overlap by 47 of their 48 hours:

```
window k      [ 24 in  | 24 out ]
window k+1     [ 24 in  | 24 out ]
```

```
window k+2      [ 24 in  | 24 out ]
                 ...
```

A window is kept only if all 48 of its hours lie inside one period; the 47 windows per site that would straddle each period boundary are discarded. This is why a period of H hours yields $H - 47$ windows (for example 45,412 − 47 = 45,365 training windows per site). Windows never mix two sites. The temporal order of the 48 hours inside a window is always preserved, by construction, whatever happens to the window afterwards.

2.4 Blocks and replicates (bootstrap only)

The principle is easiest to see on a training set only ten weeks long. *Resampling* builds a new training set of the same size by drawing weeks at random and with replacement, as when drawing numbered balls from a bag and returning each ball after reading it. The result of one such draw is a *replicate*:

```
original training data:  W1  W2  W3  W4  W5  W6  W7  W8  W9  W10
replicate 1:             W3  W7  W7  W1  W9  W2  W3  W10 W5  W8    (W4, W6 missing; W3, W7 twice)
replicate 2:             W6  W1  W4  W4  W10 W2  W8  W8  W5  W1    (W3, W7, W9 missing)
```

One network is trained on each replicate, so each network has learned from a slightly different history: the network trained on replicate 1 has never seen the weather of weeks 4 and 6. Where the networks' forecasts disagree, the forecast depends on which history the model happened to learn from.

Whole weeks are drawn, rather than single hours, because neighbouring hours are nearly identical. If single hours were drawn, a missing hour would in effect be replaced by its near-identical neighbour, and every replicate would carry practically the same information. Drawing weeks means that whole weather episodes are genuinely absent from a replicate, so the networks differ in a meaningful way.

In the actual procedure the role of a "week" is played by a *block* of 168 consecutive training windows, which may start at any hour rather than on a fixed weekday. The bootstrap does not cut the series again and does not create new windows. It takes the 45,365 training windows of one site, in time order, and draws *blocks* of $\ell = 168$ consecutive windows from them:

```
training windows of one site, in time order:   w1  w2  w3  ...  w45365

one block starting at w_j:   w_j, w_j+1, ..., w_j+167
   = 168 forecast origins spanning 7 days (215 hours of data, inputs and targets included)

one replicate for this site: [block at w_9120][block at w_301][block at w_40007] ...
   = 271 randomly placed blocks, joined and cut to exactly 45,365 windows
```

The same is done independently for the other four sites, and the five resampled sets are pooled into one replicate. Repeating this $B = 8$ times gives eight replicates, and one network is trained on each.

What the blocks accomplish needs stating precisely. Windows inside a replicate are shuffled into mini-batches during training, and the network carries no state from one window to the next, so the order in which blocks are joined plays no role. What the blocks control is *which* windows enter a replicate, and how often: neighbouring windows, which share most of their hours and the same weather, are included or left out together, a week at a time. A replicate is therefore a resampling of weeks of weather rather than of isolated hours, which is the dependence an independent (window-by-window) bootstrap would ignore. A replicate contains about 61–65% of the distinct training windows, some of them several times over.

2.5 Passes (MC Dropout only)

Dropout is a standard training technique: at every training step a random 30% of the network's neurons are switched off, so that the network cannot rely too heavily on any single one. Ordinarily dropout is switched off when forecasts are made. MC Dropout keeps it switched on. The same trained network is then given the same test window M = 100 times, and each time a different random 30% of its neurons is switched off, giving 100 slightly different 24-hour forecasts:

```
network 1, same test window, pass 1:    12:00 -> 640 W/m²
network 1, same test window, pass 2:    12:00 -> 710 W/m²
network 1, same test window, pass 3:    12:00 -> 590 W/m²
...                                      (100 passes)
```

If these forecasts stay close together, what the network has learned about this situation is robust; if they scatter, it is not. MC Dropout involves no new data, no resampling and no retraining, and it uses no periods or blocks: it runs only at prediction time, on the ordinary test windows, each used once and in no particular order. With B = 8 networks, every test window receives 8 × 100 = 800 forecasts.

For each forecast hour, the 800 values are then summarised. The point forecast is their mean, and the 95% prediction interval runs from the 2.5th to the 97.5th percentile: the 800 values are sorted and the lowest 2.5% and highest 2.5% are cut off. For the example in Figure 2(d), 12:00 on 17 April 2026 in Ankara, this gives a forecast of 659 W/m² and an interval of 332–972 W/m².

2.6 How the pieces combine

Figure 1 summarises the whole procedure in symbols.

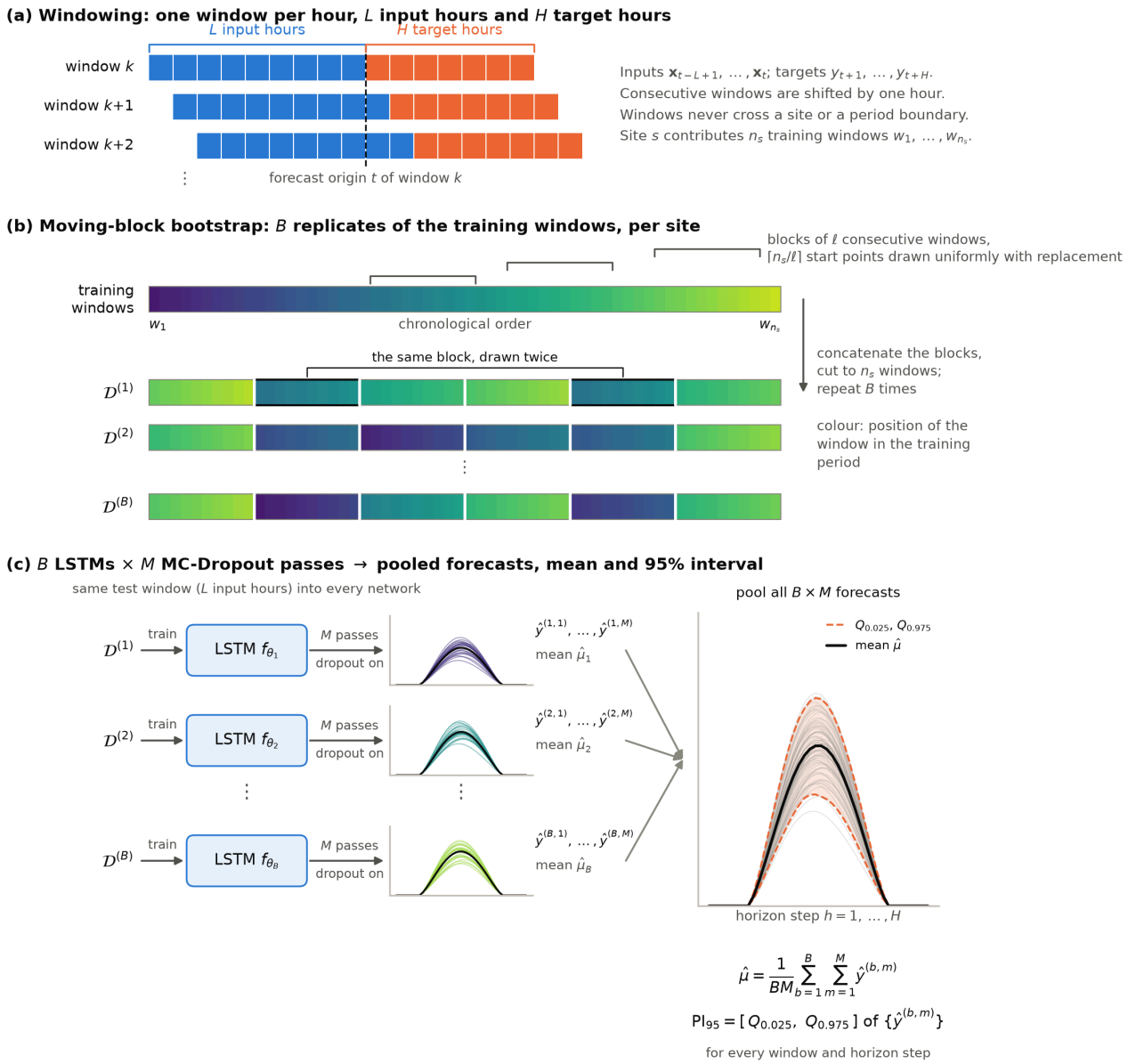


Figure 1. Overview of the forecasting and uncertainty-quantification procedure. (a) Each site's hourly series is cut into overlapping windows of L input hours (blue) followed by the H hours to be forecast (orange); a new window starts every hour. (b) Moving-block bootstrap: for each site, block start points are drawn uniformly with replacement from its n_s chronologically ordered training windows; the blocks of ℓ consecutive windows are concatenated and cut to n_s windows, giving replicate $\mathcal{D}^{(b)}$. Colour encodes each window's position in the training period, so the order is preserved within a block but not between blocks; a block can be drawn more than once (outlined) and some windows are left out. The replicates of all sites are pooled, and the procedure is repeated B times. (c) One LSTM is trained on each replicate. Each trained network is run M times on the same test window with dropout kept active, giving forecasts $\hat{y}^{(b,1)}$, …, $\hat{y}^{(b,M)}$ over the H horizon steps. The $B \times M$ forecasts are pooled; their mean is the point forecast $\hat{\mu}$ and their empirical 2.5th and 97.5th percentiles bound the 95% prediction interval, separately for every window and horizon step. Curves are illustrative.

Table 2. The two sampling mechanisms compared.

	Resampling (bootstrap)	**MC Dropout**
What is varied	The training data	The network's active neurons
When	Before training	At prediction time

	Resampling (bootstrap)	MC Dropout
Which data	Training period only	Test windows (and validation windows, if recalibrating)
How many	8 replicates, hence 8 networks	100 passes per network
Question it answers	Would the forecast change with a different training history?	Is this network's forecast stable?

Both mechanisms measure uncertainty in the model only. Neither represents the natural variability of the weather itself, such as a cloud arriving that no model could anticipate, so the resulting interval is expected to be too narrow; Section 8.5 describes the correction.

Figure 2 shows the same steps on the real data, using the replicates and trained networks of the reference run.

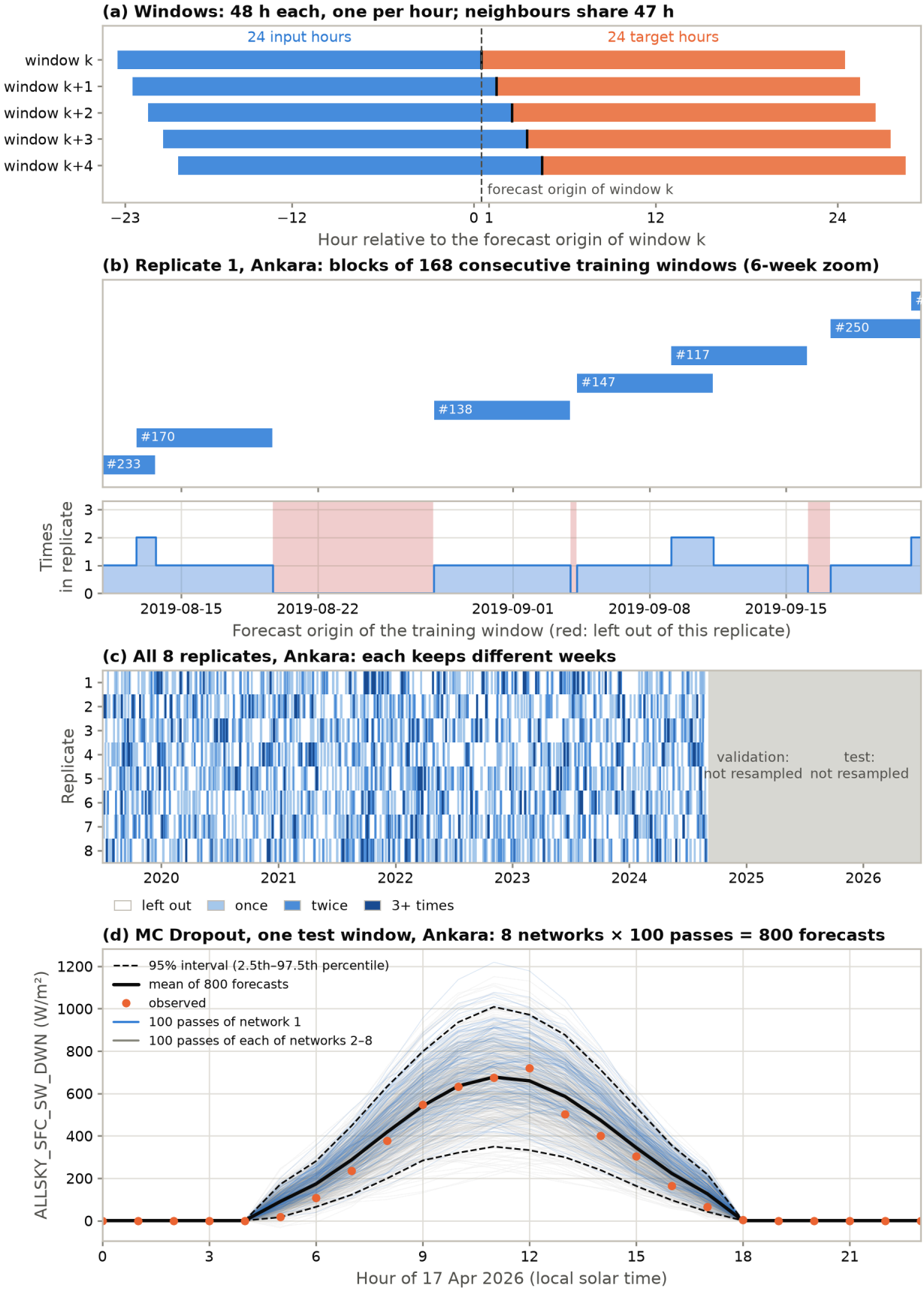


Figure 2. How the data are sampled. (a) Windows: each window holds 24 input hours (blue) and the 24 hours to be forecast (orange); a new window starts every hour, so neighbouring windows share 47 of their 48 hours. (b) Bootstrap blocks for one site in replicate 1, over six weeks of the training period. Each bar is one drawn block of 168 consecutive training windows, labelled by the order in which it was drawn; the lower plot counts how of-

ten each window appears in the replicate. Windows covered by no block (red) are left out of this replicate, and where two blocks overlap, the windows appear twice. (c) The same count for all eight replicates over the whole record, one window per day: every replicate keeps a different set of weeks, and the validation and test periods are never resampled. (d) MC Dropout on one test window: each of the eight trained networks is run 100 times with fresh dropout masks (blue: the 100 passes of network 1; grey: networks 2–8). The 800 forecasts give the mean (solid) and the 95% interval (dashed); forecasts are zero when the sun is below the horizon.

3. Problem formulation

Let $s \in \{1, \ldots, 5\}$ index the site and t the hour, expressed in each site's local solar time. Let $\mathbf{x}_{s,t} \in \mathbb{R}^{13}$ be the standardised feature vector of site s at hour t (Section 4) and $y_{s,t}$ the observed irradiance. The input of the window with origin t is

$$X_{s,t} = (\boldsymbol{x}_{s,t-23}, \ldots, \boldsymbol{x}_{s,t}),$$

and its target is the irradiance over the following 24 hours,

$$y_{s,t} = (y_{s,t+1}, \ldots, y_{s,t+24}).$$

The model maps the window and the site identity to all 24 values at once (direct multi-output forecasting):

$$\hat{y}_{s,t} = f_\theta(X_{s,t}, s) \in \mathbb{R}^{24}. \tag{1}$$

Each horizon step $h = 1, \ldots, 24$ is therefore a separate output of the same network rather than the result of feeding earlier predictions back as inputs. Only observed quantities enter the window; no numerical weather prediction or other forecast of the meteorological variables is used, so the inputs are all available at the time the forecast is issued.

A single model is trained on all five sites jointly. Site identity enters only through a learned embedding (Section 6), so information can be shared across climates while the network can still represent site-specific behaviour.

4. Input variables and feature engineering

Input set. Each hour of a site's record is described by thirteen input features (Table 3): the lagged target, six meteorological variables, and six features derived from circular quantities. The meteorological variables are air temperature (T2M), relative humidity (RH2M), dew/frost point (T2MDEW), surface pressure (PS), wind speed (WS2M) and bias-corrected precipitation (PRECTOTCORR), all at 2 m. They enter the model in their native units, with no further transformation. Only observed quantities are used, so every input is available when the forecast is issued.

Autoregressive component. All-sky surface shortwave irradiance (ALLSKY_SFC_SW_DWN) is both the forecast target and an input. Its values over the 24-hour input window give the network the recent cloudiness and the current state of the diurnal cycle. This makes the model autoregressive in irradiance. Because all 24 horizon steps are predicted at once (Section 3), earlier forecasts are never fed back as inputs.

Circular encoding of wind direction. Wind direction is an angle, so a linear representation would put 359° and 1° at opposite ends of the scale. Wind direction θ is therefore replaced by the pair (sin θ, cos θ). This maps each direction to a point on the unit circle and removes the 0°/360° discontinuity.

Cyclical time encoding. The diurnal and annual cycles are encoded in the same way. The hour of day h becomes $(\sin 2\pi h/24, \cos 2\pi h/24)$, and the day of year d becomes $(\sin 2\pi d/365.25, \cos 2\pi d/365.25)$. With this encoding 23:00 and 00:00, and 31 December and 1 January, are neighbours in feature space. The year length of 365.25 days absorbs leap years. Timestamps are in each site's local solar time, so a given value of the hour encoding matches approximately the same solar hour angle at every site.

Solar geometry as a mask, not a predictor. Two geometric quantities are computed from each site's coordinates and the timestamp alone: the apparent solar elevation at the midpoint of each hour, and the top-of-atmosphere irradiance on a horizontal surface. Neither is a model input. They define the night-time constraint on the forecasts (Section 9) and the daylight evaluation subset (Section 10.1). Excluding them keeps the input set free of information the cyclical time encodings already carry, and keeps the definition of the evaluation subset separate from what the model learns.

Site identity. All five sites are modelled by one network. Site identity is not a hand-crafted feature: each site gets a learned four-dimensional embedding vector, which is appended to the feature vector at every hour of the window, giving seventeen inputs per hour (Section 6). This lets the network share structure across climate zones while still representing site-specific behaviour. All thirteen features, the target included, are standardised using statistics from the training period only (Section 5).

Table 3. Model inputs.

Variable	Description	Unit
ALLSKY_SFC_SW_DWN	All-sky surface shortwave downward irradiance (lagged target)	W/m²
T2M	Air temperature at 2 m	°C
RH2M	Relative humidity at 2 m	%
T2MDEW	Dew/frost point at 2 m	°C
PS	Surface pressure	kPa
WS2M	Wind speed at 2 m	m/s
PRECTOTCORR	Bias-corrected precipitation	mm/h
WD2M (sin, cos)	Wind direction at 2 m, sine and cosine	–
Hour of day (sin, cos)	2π · hour / 24	–
Day of year (sin, cos)	2π · day of year / 365.25	–

5. Data partitioning, standardisation and windowing

The record (30 June 2019 00:00 – 29 June 2026 23:00, 61,368 hours per site) is divided into the three periods of Section 2.2, in proportions 74% / 11% / 15%. The boundaries are computed once and are identical at every site. The test period spans 384 days and covers all four seasons; the validation period contains no July or August and only the first eleven days of June.

Every variable, the target included, is standardised to zero mean and unit variance, with means and standard deviations estimated from the training period only and then applied unchanged to the validation and test periods. For the target these are $\bar{y}$ = 198.5 W/m² and s_y = 279.7 W/m². Networks are trained on the standardised target, and their forecasts are transformed back to W/m² before evaluation.

Windows are built as described in Section 2.3. Across the five sites this gives 226,825 training, 33,515 validation and 45,795 test windows.

6. Network architecture

The network has three parts.

- **Site embedding.** Each site is mapped to a learned 4-dimensional vector, which is concatenated to the feature vector at every hour of the window, giving 17 inputs per hour.

- **Recurrent encoder.** Two stacked LSTM layers with 64 hidden units each read the 24 input hours in order. Dropout (rate $p = 0.3$) is applied between the two layers, and again to the hidden state of the second layer at the last input hour, which summarises the window.
- **Output head.** A fully connected layer with 32 units and ReLU activation, followed by dropout ($p = 0.3$) and a linear layer with 24 outputs, one per horizon step.

The network has 57,420 trainable parameters. It contains no batch normalisation, because MC Dropout runs the network in training mode at prediction time, where batch normalisation would use the statistics of the prediction batch rather than fixed ones. Dropout is therefore the only source of randomness at prediction time.

7. Training

Each network is trained on its replicate by minimising, over mini-batches of N windows, the mean squared error on the standardised target plus a penalty on negative standardised forecasts:

$$\mathcal{L}(\theta) = (1/24N)\, \Sigma_i \Sigma_h\, (\tilde{z}_{i,h} - z_{i,h})^2 + \lambda\, (1/24N)\, \Sigma_i \Sigma_h\, max(0, -\tilde{z}_{i,h})^2, \tag{2}$$

where $z = (y - \bar{y})/s_y$ is the standardised observation, $\tilde{z}$ the standardised forecast, and $\lambda = 0.1$. The squared error targets the conditional mean, so the point forecast is an estimate of expected irradiance.

The penalty is intended to discourage physically impossible negative irradiance. Because it is evaluated on the standardised scale, however, it acts on forecasts below the training-period mean, $\hat{y} < \bar{y} = 198.5$ W/m², rather than below 0 W/m². This covers about 66% of training hours, including every night hour, so the penalty pulls low forecasts upward, towards the mean. At night the resulting bias is removed by the constraint of Section 9; on low-irradiance daylight hours (early morning, late afternoon, overcast conditions) it is not. The physically intended form would be $\max(0, -(\tilde{z} + \bar{y}/s_y))^2$, which is zero for every non-negative forecast. **[Known issue: the implemented form is the one in Eq. 2; correcting it requires retraining every network.]**

Optimisation uses Adam with an initial learning rate of 10^{-3} and mini-batches of 128 windows, reshuffled every epoch. After each epoch the loss of Eq. 2, penalty included, is computed on the validation windows with dropout disabled. The learning rate is halved when this validation loss has not improved for 7 epochs. Training stops when it has not improved for 10 epochs, or after 100 epochs, and the weights from the epoch with the lowest validation loss are kept. Every network uses the same, unresampled validation windows.

8. Uncertainty quantification

8.1 Moving-block bootstrap

Meteorological and irradiance series are strongly autocorrelated, so resampling windows independently would treat near-identical neighbouring windows as independent evidence. A moving-block bootstrap is used instead (Section 2.4). For site s with n_s chronologically ordered training windows and block length ℓ, $\lceil n_s/\ell \rceil$ block start positions are drawn uniformly and with replacement from $\{1, \ldots, n_s - \ell + 1\}$. The blocks of ℓ consecutive windows are joined and cut to exactly n_s windows. This is repeated for every site, and the five resampled sets are pooled into one replicate:

$$\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \ldots, \mathcal{D}^{(B)}. \tag{3}$$

With $n_s = 45{,}365$ and $\ell = 168$, each site contributes 271 blocks per replicate. Resampling site by site keeps the number of windows per site identical in every replicate, so no replicate is dominated by one climate. Each window carries its own inputs and targets, so resampling cannot pair an input with the wrong target. The block

length of 168 windows (one week of forecast origins) is chosen to span the diurnal cycle and the multi-day lifetime of synoptic weather systems. As in any moving-block bootstrap, windows within $\ell - 1$ of either end of a site's training series can be drawn by fewer block positions and so appear less often.

Only training windows are resampled. Validation windows are the same for every replicate and are used only for early stopping and learning-rate scheduling; test windows take no part in training or resampling.

[To be decided: whether to run a block-length sensitivity analysis (ℓ = 24, 72 and 168 windows); it has not been run.]

8.2 Monte Carlo Dropout

One network f_{θ_b} is trained on each replicate $\mathcal{D}^{(b)}$, starting from a different random initialisation. During training, dropout acts as a regulariser. At prediction time it is kept active, so each forward pass applies a fresh random dropout mask $\mathbf{z}_{b,m}$, drawn independently for every window. With M passes per network, every input window receives $B \times M$ stochastic forecasts (Section 2.5):

$$\hat{y}^{(b,m)} = f_{\theta_b}(X, s; \mathbf{z}_{b,m}), \quad b = 1, \ldots, B, \; m = 1, \ldots, M. \tag{4}$$

This study uses $B = 8$ and $M = 100$, giving 800 forecasts for every window and horizon step. All quantities below are computed separately for each element, i.e. for each window and horizon step.

8.3 Predictive mean and variance decomposition

The point forecast is the mean over all $B \times M$ forecasts:

$$\hat{\mu} = (1/BM) \, \Sigma_{b=1}^{B} \, \Sigma_{m=1}^{M} \, \hat{y}^{(b,m)}. \tag{5}$$

The spread of the pooled forecasts has two sources. Writing the mean of network b as

$$\hat{\mu}_b = (1/M) \, \Sigma_{m=1}^{M} \, \hat{y}^{(b,m)}, \tag{6}$$

the variability within each network due to dropout is

$$\sigma^2_{MC} = (1/B) \, \Sigma_{b=1}^{B} \, (1/M) \, \Sigma_{m=1}^{M} \, (\hat{y}^{(b,m)} - \hat{\mu}_b)^2, \tag{7}$$

and the variability between networks due to the bootstrap replicates is

$$\sigma^2_{boot} = (1/B) \, \Sigma_{b=1}^{B} \, (\hat{\mu}_b - \hat{\mu})^2. \tag{8}$$

By the law of total variance, their sum is exactly the variance of the pooled forecasts:

$$\sigma^2_{epi} = \sigma^2_{MC} + \sigma^2_{boot} = (1/BM) \, \Sigma_b \, \Sigma_m \, (\hat{y}^{(b,m)} - \hat{\mu})^2. \tag{9}$$

Both components describe uncertainty about the model, that is, epistemic uncertainty: how much the conditional-mean forecast would change with different training data (σ^2_{boot}) and under the approximate posterior over network weights that MC Dropout represents (σ^2_{MC}). Reporting the two separately shows how much each part of the ensemble contributes. **[Known issue: the pipeline currently stores only the total, σ_{epi}; the two components require a small change to be reported.]**

8.4 Prediction interval

The nominal 95% prediction interval is given by the empirical 2.5th and 97.5th percentiles of the $B \times M$ pooled forecasts:

$$PI_{95} = [Q_{0.025}(\{\hat{y}^{(b,m)}\}), \, Q_{0.975}(\{\hat{y}^{(b,m)}\})]. \tag{10}$$

Because the interval is taken from the pooled sample rather than from a Gaussian assumption, it can be asymmetric, which arises naturally when the networks disagree in one direction.

8.5 Aleatoric uncertainty and interval recalibration

Every network outputs a single value per horizon step, so the pooled forecasts describe uncertainty about the conditional mean only. They contain no term for the variability of the observation around that mean (aleatoric uncertainty), which in irradiance is dominated by cloud processes that a 24-hour history cannot resolve. The pooled interval is therefore expected to cover fewer than 95% of observations.

This can be corrected by split-conformal recalibration on the validation period. The pooled forecasts for each element are rescaled about their own mean,

$$\hat{y}^{(b,m)} \to \hat{\mu} + k\,(\hat{y}^{(b,m)} - \hat{\mu}), \tag{11}$$

so the point forecast, and hence every point-accuracy metric, is unchanged, while the interval endpoints and the whole predictive distribution scale together. To estimate k, every network is also run M times on each validation window, exactly as on the test windows. For each validation element with observation y and interval $[L, U]$, the conformity score is the smallest factor that would have covered it:

$$r = (y - \hat{\mu}) / (U - \hat{\mu}) \text{ if } y \geq \hat{\mu}, \quad r = (\hat{\mu} - y) / (\hat{\mu} - L) \text{ otherwise.} \tag{12}$$

Given n scores, k is the $\lceil (n + 1)(1 - \alpha) \rceil$-th smallest, with $\alpha = 0.05$. Factors are estimated separately for each cell of a grid defined by **[to be decided: the grid, e.g. site × season]**; cells with fewer than 200 usable scores fall back to the pooled factor. Night-time elements, where forecast and observation are both zero, carry no information about the interval width and are excluded from calibration. The reference run reported here uses no recalibration.

Two limitations follow from the data. First, the validation period also serves for early stopping, so the calibration data are not entirely independent of training decisions; the dependence is weak, because early stopping selects only the epoch. Second, the validation period contains no July or August and only the first eleven days of June, so the summer factors are extrapolated from the other months.

9. Physical constraint at night

Daylight is defined geometrically: an hour is a daylight hour if the apparent solar elevation at the midpoint of the hour, computed from the site coordinates and the timestamp, is above zero. When the sun is below the horizon, irradiance is physically zero. For every such element, all $B \times M$ forecasts are set to zero, so the point forecast and both interval endpoints are zero. The constraint uses only site and time, so it is available when the forecast is issued, and it is applied identically to every model.

10. Evaluation

10.1 Evaluation subsets

All metrics are computed on the test period, over every test window and all 24 horizon steps, and reported for two subsets: all hours, and daylight hours only (solar elevation > 0). About half of all elements are night hours whose observation and (constrained) forecast are both exactly zero. These lower RMSE and MAE, inflate R^2 because the day–night cycle dominates the total variance, and are covered by every interval by construction. **Daylight metrics are therefore the headline results**; all-hours metrics are given for completeness.

Because test windows overlap, each test hour appears as a target in up to 24 windows, once at each horizon step. Metrics are averages over elements, so every hour is counted once per horizon step at which it is forecast.

Results are reported pooled over the five sites, for each site, pooled over the four sites other than Rize (which forms a distinct radiative regime), and for each horizon step.

10.2 Point-forecast metrics

With $\hat{\mu}_i$ the point forecast and y_i the observation over the N elements of a subset:

$$RMSE = \sqrt{[(1/N)\,\Sigma_i\,(\hat{\mu}_i - y_i)^2]},\quad MAE = (1/N)\,\Sigma_i\,|\hat{\mu}_i - y_i|,\quad R^2 = 1 - \Sigma_i(y_i - \hat{\mu}_i)^2 \,/\, \Sigma_i(y_i - \bar{y}_{sub})^2, \tag{13}$$

where $\bar{y}_{sub}$ is the mean observation of the subset.

10.3 Probabilistic metrics

With $[L_i, U_i]$ the 95% interval and $\mu = 0.95$ the nominal coverage:

- **Coverage probability**, $CP = (1/N)\,\Sigma_i\,\mathbb{1}(L_i \le y_i \le U_i)$;
- **Reliability**, $|CP - \mu|$;
- **Mean prediction interval width**, $MPIW = (1/N)\,\Sigma_i\,(U_i - L_i)$, in W/m²;
- **Prediction interval normalised width**, $PINW = MPIW / (\max y - \min y)$, with the range taken over the subset;
- **Coverage width-based criterion**, $CWC = PINW \cdot [1 + \gamma \exp(-\eta\,(CP - \mu))]$, with $\gamma = 1$ if $CP < \mu$ and 0 otherwise, and $\eta = 50$;
- **Continuous ranked probability score**, estimated from the $S = B \times M$ pooled forecasts as

$$CRPS = (1/N)\,\Sigma_i\,[\,(1/S)\,\Sigma_j\,|\hat{y}_i^{(j)} - y_i| - (1/2S^2)\,\Sigma_j\,\Sigma_{j'}\,|\hat{y}_i^{(j)} - \hat{y}_i^{(j')}|\,], \tag{14}$$

in W/m². CRPS rewards calibration and sharpness together and reduces to MAE for a deterministic forecast, which makes it directly comparable with the reference forecasts below.

A run is judged first on whether daylight CP is close to 0.95, then on interval sharpness (PINW, CWC) and CRPS, alongside the point metrics.

10.4 Reference forecasts

Two reference forecasts are evaluated on exactly the same test windows and subsets:

- **Climatology**: the mean irradiance of the same site, calendar month and hour of day over the training period;
- **Persistence**: the observed irradiance at the same site and hour on the previous day.

A model is considered informative only if it beats climatology on daylight RMSE and R^2 and persistence on daylight MAE, the strongest reference on each measure.

11. Reproducibility

Each experiment is fully specified by a configuration file and a master random seed. The resampling generator is seeded from the master seed, and network b is initialised from seed + b, so the replicates and their initialisations are reproducible. Results are reproducible on the same hardware backend; different backends do not agree bit for bit. The reference run was trained in PyTorch on an Apple-silicon GPU (MPS backend).